

UPGRADED PROJECT DOCUMENTATION

Real-Time Word Count using Apache Kafka and PySpark Structured Streaming

Project Author: Venky

GitHub: VectorsMap-04/realtime-wordcount

1. Executive Summary

This document describes the upgraded implementation of the Real-Time Word Count project. The project was evolved into a local real-time stream-processing application using Apache Kafka as the event-streaming layer and PySpark Structured Streaming as the processing engine. Text messages are published to a Kafka topic, consumed by Spark, transformed into individual words, aggregated by frequency, and displayed as streaming results in the console.

The upgraded implementation also uses an isolated project environment containing Python 3.11.13, Java 17, PySpark 3.5.9, Apache Kafka 3.7.2, and the Spark SQL Kafka connector. The project runtime is kept inside the project directory so that it does not replace or interfere with the system Python installation.

2. Project Evolution

The original MSME project documentation is preserved separately in the repository. The upgraded implementation builds on that project by introducing a real-time streaming architecture rather than treating word counting as a simple batch-processing task.

Original approach	Upgraded approach
Word-count processing	Real-time stream processing
Static/input-oriented processing	Continuous Kafka event stream
Basic word frequency	Streaming word-frequency aggregation
Single application focus	Producer → Kafka → Spark processing pipeline
General runtime setup	Isolated Python 3.11 + Java 17 environment

3. Objectives

The main objectives of the upgraded project are to demonstrate practical real-time data streaming and stream processing concepts using technologies commonly used in data engineering.

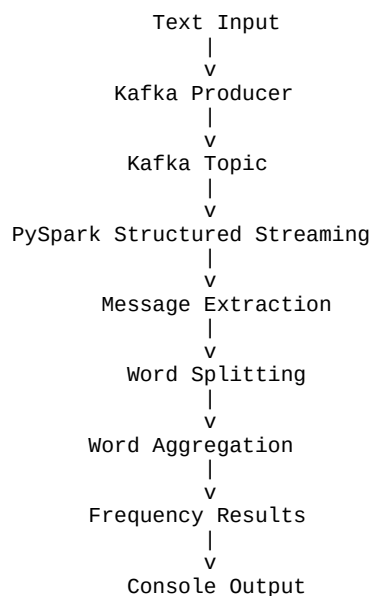
The implementation focuses on:

- Receiving continuous text events through Apache Kafka.
- Processing Kafka streams using PySpark Structured Streaming.
- Extracting and transforming message values into words.
- Calculating word frequencies continuously.
- Displaying processed results in real time.
- Maintaining isolated project-specific Python and Java runtimes.
- Providing reproducible startup scripts for the environment and services.

4. Technology Stack

Technology	Version / Configuration	Purpose
Python	3.11.13	Application runtime
Java	17.0.20.1	Runtime required by Spark
PySpark	3.5.9	Structured Streaming and processing
Apache Kafka	3.7.2	Event streaming / message broker
Kafka connector	spark-sql-kafka-0-10_2.12-3.5.9.jar	Kafka integration with Spark
Python environment	Project-local venv	Dependency isolation
OS	Ubuntu 26.04 LTS	Development environment

5. System Architecture



Kafka provides the streaming boundary between data generation and processing. Spark Structured Streaming continuously consumes the Kafka records and applies transformations to the message value. The resulting word-frequency information is written to the console sink.

6. Project Directory Structure

```
realtime-wordcount/
|
|■■■■ app/
|■■■■■■■■ realtime_wordcount.py
|■■■■ jars/
|■■■■■■■■ spark-sql-kafka-0-10_2.12-3.5.9.jar
|■■■■ documentation/
|■■■■■■■■ MSME_Original_Project_Documentation.pdf
|■■■■■■■■ Upgraded_Project_Documentation.pdf
|■■■■ activate.sh
|■■■■ requirements.txt
|■■■■ run-wordcount.sh
|■■■■ start-kafka.sh
|■■■■ start-zookeeper.sh
|■■■■ .gitignore
```

Large local runtime directories such as the project virtual environment, local Java installation, Kafka runtime/data directories, checkpoints, logs, and downloaded archives are excluded from version control through `.gitignore`. The Git repository therefore contains the source code, configuration-related scripts, required connector JAR, dependency specification, and documentation rather than the complete local runtime.

7. Environment Setup

A project-local Python 3.11.13 installation and virtual environment were created. The system Python remains separate.

```
cd ~/realtime-wordcount
source ~/realtime-wordcount/venv/bin/activate

python --version
pip --version
```

The verified environment uses Python 3.11.13 and pip 26.2.1. PySpark 3.5.9 was installed successfully inside the project virtual environment.

8. Java Configuration

```
export JAVA_HOME="$HOME/realtime-wordcount/jdk-17/java"
export PATH="$JAVA_HOME/bin:$PATH"

java -version
```

The project uses a local Temurin/OpenJDK 17 installation. The Java path is exported before starting Spark so the application uses the project-specific Java runtime.

9. Apache Kafka Setup

Apache Kafka 3.7.2 was extracted into the project directory. The Kafka server configuration was updated so its log directory points to a project-local location: `/home/venky/realtime-wordcount/kafka-data`.

ZooKeeper and Kafka were configured and tested successfully in the local environment. Kafka topics can then receive producer messages for the Spark application.

10. Spark–Kafka Integration

The initial Spark application produced a 'Failed to find data source: kafka' exception. This was resolved by providing the Spark SQL Kafka connector JAR compatible with Spark 3.5.9.

```
jars/
■■■ spark-sql-kafka-0-10_2.12-3.5.9.jar
```

After the connector was made available to the Spark application, the streaming application successfully connected to Kafka and processed incoming messages.

11. Application Workflow — Step by Step

Step 1 — Activate the environment

The project startup script activates the local Python virtual environment and configures Java 17.

Step 2 — Start ZooKeeper

The ZooKeeper service is started for the Kafka 3.7.2 setup used by the project.

Step 3 — Start Kafka

Kafka is started using the project startup script and its local data directory.

Step 4 — Start a Kafka producer

Text messages are entered into a Kafka topic through a Kafka console producer.

Step 5 — Start the Spark application

run-wordcount.sh launches the PySpark Structured Streaming application.

Step 6 — Read Kafka records

Spark reads Kafka records and obtains the message value from each event.

Step 7 — Transform text

The application converts each message into individual words.

Step 8 — Aggregate frequencies

Words are grouped and their occurrences are aggregated by the streaming computation.

Step 9 — Display results

The processed word-frequency results are written to the console.

12. Example Data Flow

Example input messages:

```
hello world
hello kafka
kafka spark
```

The stream-processing logic identifies the words and aggregates their frequencies. An example result is:

```
hello    2
kafka    2
world    1
spark    1
```

The exact console presentation depends on the streaming query configuration and the data received. The important result is the continuously updated word-frequency aggregation.

13. Environment Verification

The following checks were completed during implementation:

- Python 3.11.13 was verified inside the virtual environment.
- Java 17.0.20.1 was verified using the project-local JAVA_HOME.
- PySpark 3.5.9 was installed and imported successfully.

- A SparkSession test completed successfully and reported Spark version 3.5.9.
- Apache Kafka 3.7.2 was verified using kafka-topics.sh --version.
- The Spark Kafka data-source integration was corrected using the compatible connector JAR.
- The final streaming application displayed word counts from Kafka input.

14. Problems Encountered and Resolutions

Issue	Resolution
System Python was 3.14.4 while the project required 3.13.4	Installed Python 3.13 locally and created a project virtual environment.
Spark initially could not find the Kafka data source.	Added the Spark SQL Kafka connector JAR compatible with Spark 3.5.9.
Kafka/ZooKeeper startup required correct local configuration	Created project startup scripts and configured Kafka log storage under the project directory.
GitHub HTTPS password authentication failed.	Authenticated GitHub CLI using browser/device authentication and then pushed the main branch.
Runtime files could unnecessarily enter Git.	Used .gitignore to exclude local runtime/data directories and other generated files.

15. GitHub Version Control

The project was initialized as a Git repository, the default branch was renamed to main, and the project source/configuration files were committed and pushed to the GitHub repository VectorsMap-04/realtime-wordcount.

The repository contains the project implementation and documentation while excluding large/generated local runtime data through .gitignore.

16. Security and Reproducibility Notes

Authentication credentials and GitHub tokens should never be placed in source code, README files, or committed configuration files. The GitHub CLI authentication used during setup remains outside the repository.

The project-local environment improves reproducibility because the application can use fixed versions of Python, Java, PySpark, and Kafka rather than depending on whatever versions happen to be installed system-wide.

17. Current Limitations

- The current output is console-based rather than persisted to a production database.
- The project is configured for local development rather than a multi-node production Kafka/Spark cluster.
- The current analysis is focused on word frequency rather than advanced NLP.
- Monitoring, metrics, alerting, and dashboard integration have not yet been implemented.
- The current documentation reflects the implemented local architecture; future features should be documented after they are actually built and tested.

18. Future Enhancements

- Window-based word-frequency analysis for time-bounded results.
- Top-N word detection and ranking.
- Persistent storage using PostgreSQL or another database.

- Real-time dashboards using Grafana, Power BI, or another visualization platform.
- Kafka partitioning and scalable consumer processing.
- Fault-tolerant and checkpointed production deployment.
- Docker/container-based deployment.
- Cloud deployment using managed Kafka and Spark services.
- REST API for exposing processed results.
- Real-time monitoring, metrics, logging, and alerting.
- Advanced text analytics and NLP.
- Integration with other real-world streaming data sources.

19. Conclusion

The upgraded Real-Time Word Count project demonstrates a complete local streaming pipeline using Kafka and PySpark Structured Streaming. The implementation establishes the core concepts required for real-time event ingestion, stream transformation, aggregation, and output while maintaining an isolated runtime environment.

The project also provides a practical foundation for progressing from a basic word-count application toward more advanced real-time analytics systems, including persistent storage, dashboards, scalable processing, monitoring, APIs, cloud deployment, and real-world streaming data integration.

20. Repository Documentation

The repository maintains two documentation levels: the original MSME project documentation and this upgraded project documentation. The original report is retained for historical/reference purposes, while this document records the upgraded Kafka + PySpark implementation.

Author: Venky

GitHub: <https://github.com/VectorsMap-04/realtime-wordcount>